# SAIL-PROXY User Guide - Chapter 15

st-gr

2026-09-22

# SAIL-PROXY User Guide

*Multi-provider AI Gateway for SAP AI Core* **Author:** *st-gr*

<< Previous Chapter | Content Table

---

## Tabular prediction with SAP-RPT

SAIL-PROXY relays SAP's relational pretrained transformer (RPT) models — `sap-rpt-1-small`, `sap-rpt-1-large`, `sap-rpt-1.5`, `sap-rpt-1.5-large`, `sap-rpt-1.6`, `sap-rpt-1.6-large`. These models predict values in a table, not text in a conversation: you send rows of data, some of them with one or more columns left as a placeholder, and the model fills the placeholders in. There is no training step and no fine-tuning — the model learns the pattern from the other rows in the same request, entirely in-context, and forgets it again once the request is answered.

Typical uses are classification (which category does this row belong to?) and regression (what number belongs here?) over your own tabular data — support ticket triage, churn scoring, demand estimates — without training or hosting a model of your own.

### Prerequisites

- **SAIL-PROXY installed and running** (see Installation) and a **gateway API key** (see Admin Cockpit).
- A **running deployment** of one of the six `sap-rpt-*` models in your SAP AI Core resource group, and that model in your **entitlement catalog** (see Admin Cockpit). A model with no deployment, or one outside your entitlement, is refused.

### Making a prediction

```
POST /sap/v1/rpt/{model}/predict
```

`{model}` is the model's bare name (e.g. `sap-rpt-1.6`) or its `--deployed` id — both resolve to the same deployment. Send the key as `x-api-key: <gateway API key>` (or `Authorization: Bearer <gateway API key>`, as the other routes accept).

The request body and response are SAP's own shape, passed through unchanged — see Describing your columns below for the one part worth getting right. A worked example, run against a live `sap-rpt-1.6` deployment with a synthetic support-ticket table (four known rows, one row to predict, two target columns, explanations requested):

```
curl -X POST http://localhost:3000/sap/v1/rpt/sap-rpt-1.6/predict \
  -H "x-api-key: your-api-key" \
  -H "Content-Type: application/json" \
  -d '{
    "prediction_config": {
```

```
      "target_columns": [
        { "name": "Open Days", "task_type": "regression",     "prediction_placeholder": "[PRE
        DICT]" },
        { "name": "Priority",  "task_type": "classification", "prediction_placeholder": "[PRE
        DICT]" }
      ],
      "explanations": { "top_column_scores": 3, "top_relevant_context_rows": 3 }
    },
    "index_column": "Ticket ID",
    "rows": [
      { "Ticket ID": "T-1000", "Region": "North", "Channel": "email", "Open Days": "2",  "Pri
        ority": "High" },
      { "Ticket ID": "T-1001", "Region": "South", "Channel": "phone", "Open Days": "5",  "Pri
        ority": "Medium" },
      { "Ticket ID": "T-1002", "Region": "East",  "Channel": "chat",  "Open Days": "8",  "Pri
        ority": "Low" },
      { "Ticket ID": "T-1003", "Region": "West",  "Channel": "email", "Open Days": "11", "Pri
        ority": "High" },
      { "Ticket ID": "T-2001", "Region": "North", "Channel": "chat",  "Open Days": "[PREDIC
        T]", "Priority": "[PREDICT]" }
    ]
  }'
```


```
{
  "explanations": {
    "top_column_scores": [ { "Channel": 0.551, "Region": 0.449 } ],
    "top_relevant_context_rows": [ [2, 1, 0] ]
  },
  "id": "0684d346-490a-4221-85f2-3b0e9b12f707",
  "metadata": {
    "context_mode": "default",
    "num_columns": 5,
    "num_predictions": 2,
    "num_query_rows": 1,
    "num_rows": 5
  },
  "predictions": [
    {
      "Ticket ID": "T-2001",
      "Open Days": [ { "prediction": 5.447654724121094, "confidence": null, "confidence_inter
        val": [2, 11] } ],
      "Priority":  [ { "prediction": "Low", "confidence": 1, "confidence_interval": null } ]
    }
  ],
  "status": { "code": 0, "message": "ok" }
}
```


Field by field:

- **`rows`** carries every row: the ones with real values (context) come first, and the ones to predict (query rows, carrying the placeholder in every target column) come after. `index_column` names the column that identifies a row.
- **`predictions`** has one entry per query row, keyed by its index value, with one entry per target column inside it:
  - **`prediction`** is the predicted value — a number for a regression target, a category label for a classification target.
  - **`confidence`** is set for a classification target (0–1, how sure the model is of the predicted label) and `null` for a regression target.
  - **`confidence_interval`** is set for a regression target (a `[low, high]` range) and `null` for a classification target.
- **`metadata`** describes the request the model actually saw: `num_rows` and `num_columns` are the total rows and columns sent, `num_query_rows` is how many of those rows were predicted, and `num_predictions` is `num_query_rows × number of target columns`. `context_mode` is covered under Deep context below.
- **`explanations`** is present only if you asked for it — see Explanations.

- The gateway relays the `ai-inference-id` response header, useful for correlating a call with SAP support.

## Describing your columns

An optional `data_schema` field tells the model the type of a column. It is a **dictionary**, keyed by column name, each value an object with a `dtype`:

```
"data_schema": {
  "Open Days": { "dtype": "numeric" },
  "Priority":  { "dtype": "string" }
}
```

SAP's own SDK documentation shows an array of `{"name": ..., "dtype": ...}` objects instead — that form is **rejected** with an HTTP 422 naming every column as an invalid `SchemaFieldConfig`. Use the dictionary form shown above.

## Explanations

`sap-rpt-1.5` and later support an `explanations` block in `prediction_config`:

```
"explanations": { "top_column_scores": 3, "top_relevant_context_rows": 3 }
```

`top_column_scores` returns, per prediction, the N columns that most influenced it with a 0–1 importance score each; `top_relevant_context_rows` returns the N context rows (as row indexes into the `rows` you sent) the model relied on most. `sap-rpt-1-small` does not support explanations and answers with an HTTP 422 `extra_forbidden` if you ask for them.

## Deep context

The large models (`sap-rpt-1-large`, `sap-rpt-1.5-large`, `sap-rpt-1.6-large`) accept `"context_mode": "deep"` in `prediction_config`, which lets the model draw on a larger context table at a higher price — see Metering below. Every other model accepts only the default `"context_mode": "default"` (the same as omitting the field) and answers with an HTTP 422 if `"deep"` is requested. A response's `metadata.context_mode` says which mode actually ran; `sap-rpt-1-small` omits `context_mode` from its response metadata entirely.

## Limits

SAP's models enforce their own limits; the gateway does not re-check them, and any violation comes back as SAP's own error, unchanged. Measured limits: at most 10 target columns and 128 query rows per request; the context rows before the query rows cap out at 2,048 or 65,536 depending on the model, and columns at 100 or 256.

The gateway itself caps a request body at 10 MB — a request larger than that is refused before it ever reaches SAP.

Two error shapes can come back, both in the same `detail`/`status` envelope. The gateway's own refusals — an unknown model, for instance:

```
{
  "detail": [
    { "loc": [], "msg": "Model sap-rpt-9 is not available", "type": "model_not_found" }
  ],
  "status": { "code": 2, "message": "Invalid input" }
}
```

And SAP's own errors, such as eleven target columns:

```
{
  "detail": [
    { "loc": ["prediction_config", "target_columns"],
      "msg": "Value error, target_columns may not contain more than 10 entries",
      "type": "value_error" }
  ],
  "id": "…",
  "status": { "code": 2, "message": "Invalid input" }
}
```

SAP's own error responses — including SAP's own authentication and rate-limit errors — are returned exactly as SAP sent them, never rewritten by the gateway.

## Metering

Usage is measured in **cells**, not tokens: every cell of data you send (every row × every column, index and target columns included) counts as an input cell, and every value the model predicts (query rows × target columns, `num_predictions` in the response) counts as a predicted cell. Both figures come from the response, so a request SAP refuses bills nothing.

The Admin Cockpit's Usage Analytics shows these models' usage in cells, and their Model Library entries price cells rather than tokens. A tokens-per-day quota still applies to these models — it counts the cells.

## Parquet

`POST /sap/v1/rpt/{model}/predict-parquet` also exists: the gateway relays it unchanged, exactly like the JSON route, including its usage accounting. Whether a given deployment actually serves it is up to SAP — the deployments used to write this guide did not, both answering HTTP 404 — so no worked example is given here.

---

*For general troubleshooting, see the Troubleshooting guide or the FAQ.*